



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Karina García Morales

Asignatura: Fundamentos de Programación

Grupo: 22

No de Práctica(s): Practica 10

Integrante(s): Roja Martínez Francisco Alexander

*No. de lista o
brigada:* 40

Semestre: 2026-1

Fecha de entrega: 11 de noviembre de 2025

Observaciones:

CALIFICACIÓN:

Arreglos multidimensionales

Objetivo:

El alumno utilizará arreglos de dos dimensiones en la elaboración de programas que resuelvan problemas que requieran agrupar datos del mismo tipo, en estructuras que utilicen dos índices.

Introducción

Un arreglo de dos dimensiones (o matriz) es una estructura de datos que almacena una colección fija y contigua de elementos del mismo tipo. Piensa en él como una cuadrícula o tabla.

Para localizar cualquier dato dentro de esta "tabla", necesitas obligatoriamente dos índices: uno para la fila y otro para la columna.

La principal ventaja de usar arreglos es que mejoran la claridad y la eficiencia de tu código. Al agrupar datos relacionados y del mismo tipo bajo un solo nombre, puedes manipularlos de forma mucho más organizada y efectiva en tu programa.

Arreglos Multidimensionales

El lenguaje C ofrece la capacidad de definir arreglos con múltiples dimensiones. Su sintaxis es clara y estructurada:

tipoDato nombre[tamaño][tamaño]...[tamaño];

Esta estructura nos permite agrupar datos de este tipoDato (como enteros, flotantes o estructuras) bajo un único nombre de identificador. El número de corchetes define la dimensionalidad del arreglo, y cada tamaño especifica el límite de elementos para su respectiva dimensión.

¿Qué significa cada cosa?

- tipoDato: Simplemente es el tipo de dato que vamos a guardar (números enteros, decimales, letras, etc.).

- nombre: Es el nombre que le damos al arreglo (la variable).
- [tamaño]: Cada par de corchetes representa una dimensión. El número que pongamos ahí dentro es el límite máximo de elementos que puede tener esa dimensión. Si ponemos tres pares de corchetes, tenemos un arreglo de tres dimensiones

Apuntadores y su relación con arreglos de dos dimensiones

Para una manipulación eficiente de datos, es esencial entender que un apuntador es una variable cuyo contenido es la dirección de memoria de otra variable, actuando como una referencia directa a su ubicación. Al trabajar a este nivel, los apuntadores garantizan un acceso y procesamiento de datos sumamente rápido.

Manejo Básico de Apuntadores

La declaración de un apuntador se identifica por el asterisco (*):

TipoDeDato *apuntador, variable;

Para asignar una dirección de memoria a este apuntador, se utiliza el operador ampersand (&), que devuelve la dirección de la variable (como ocurre implícitamente al usar scanf):

apuntador = &variable;

Es una regla estricta que los apuntadores deben coincidir en el tipo de dato con la variable referenciada. Finalmente, para acceder al valor almacenado en esa dirección (operación de desreferencia), se antepone nuevamente el asterisco (*) al nombre del apuntador.

Ejercicios en clase

¿Qué es un arreglo bidimensional?

Un arreglo bidimensional (o matriz) es una estructura de datos homogénea que organiza los elementos en dos dimensiones distintas: filas y columnas.

- Estructura Lógica: Se visualiza como una tabla o cuadrícula, donde para acceder a un dato específico se requieren dos índices: uno para la fila y otro para la columna, utilizando la notación arreglo[fila][columna].
- Propósito: Permite la manipulación eficiente de conjuntos de datos relacionados que tienen una estructura tabular (como coordenadas, tableros de juego, o datos financieros).
- Almacenamiento en Memoria: A pesar de la estructura lógica de tabla, los elementos se guardan de forma contigua y secuencial en la memoria del sistema (por filas), lo cual es clave para el uso eficiente de apuntadores.

Realiza un ejemplo del uso de apuntadores en un arreglo bidimensional

```
int lista[10][10], *apuntador;  
apuntador= &lista[i][j]; apuntador=*lista;
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int lista[10][10];
```

```
    int *apuntador;
```

```
    int k;
```

```
// 1. Inicializar la matriz con valores (0 a 99)
```

```
for (int i = 0; i < 10; i++) {
```

```
    for (int j = 0; j < 10; j++) {
```

```
        lista[i][j] = i * 10 + j;
```

```
    }
```

```
}
```

```
// 2. Asignar el apuntador al inicio de la matriz (Equivalente a: apuntador =  
&lista[0][0];)
```

```
apuntador = *lista;
```

```
// 3. Recorrer la matriz completa (100 elementos) usando solo el apuntador
```

```
printf("Recorrido secuencial usando apuntador:\n");
```

```
for (k = 0; k < 100; k++) {
```

```
    // Acceder al valor con *(apuntador + k) e imprimirlo
```

```
    printf("%d ", *(apuntador + k));
```

```
    // Formato para mostrar 10 valores por linea (simulando filas)
```

```
    if ((k + 1) % 10 == 0) {
```

```
        printf("\n");
```

```
    }
```

```
}
```

```
return 0;
```

```
}
```

```
main.c
1 //Roja Martinez Francisco Alexander
2 //Actividad
3
4 #include <stdio.h>
5
6 int main() {
7     int lista[10][10];
8     int *apuntador;
9     int k;
10
11     // 1. Inicializar la matriz con valores (0 a 99)
12     for (int i = 0; i < 10; i++) {
13         for (int j = 0; j < 10; j++) {
14             lista[i][j] = i * 10 + j;
15         }
16     }
17
18     // 2. Asignar el apuntador al inicio de la matriz (Equivalente a: apuntador = &lista[0][0];)
19     apuntador = *lista;
20
21     // 3. Recorrer la matriz completa (100 elementos) usando solo el apuntador
22     printf("Recorrido secuencial usando apuntador:\n");
23     for (k = 0; k < 100; k++) {
24
25     }
26 }
```

input

Recorrido secuencial usando apuntador:

```
0 1 2 3 4 5 6 7 8 9
10 11 12 13 14 15 16 17 18 19
20 21 22 23 24 25 26 27 28 29
30 31 32 33 34 35 36 37 38 39
40 41 42 43 44 45 46 47 48 49
50 51 52 53 54 55 56 57 58 59
```

1.- Del programa 1a.c modificar para que se pidan los datos por el usuario, de la matriz de 3 por 3

```
main.c
1 // Roja Martinez Francisco Alexander
2 // Programala.c
3
4
5 #include <stdio.h>
6
7 int main() {
8
9     int matriz[3][3];
10     int i, j;
11
12     printf("Introduce los 9 valores de la matriz (3x3):\n");
13
14     for (i = 0; i < 3; i++) { // filas
15         for (j = 0; j < 3; j++) { // columnas
16             printf("Introduce el valor para la posición [%d][%d]: ", i, j);
17             scanf("%d", &matriz[i][j]);
18         }
19     }
20
21     printf("\nImprimir Matriz\n");
22     for (i = 0; i < 3; i++) { //filas
23         for (j = 0; j < 3; j++) { // columnas
24
25         }
26     }
27 }
```

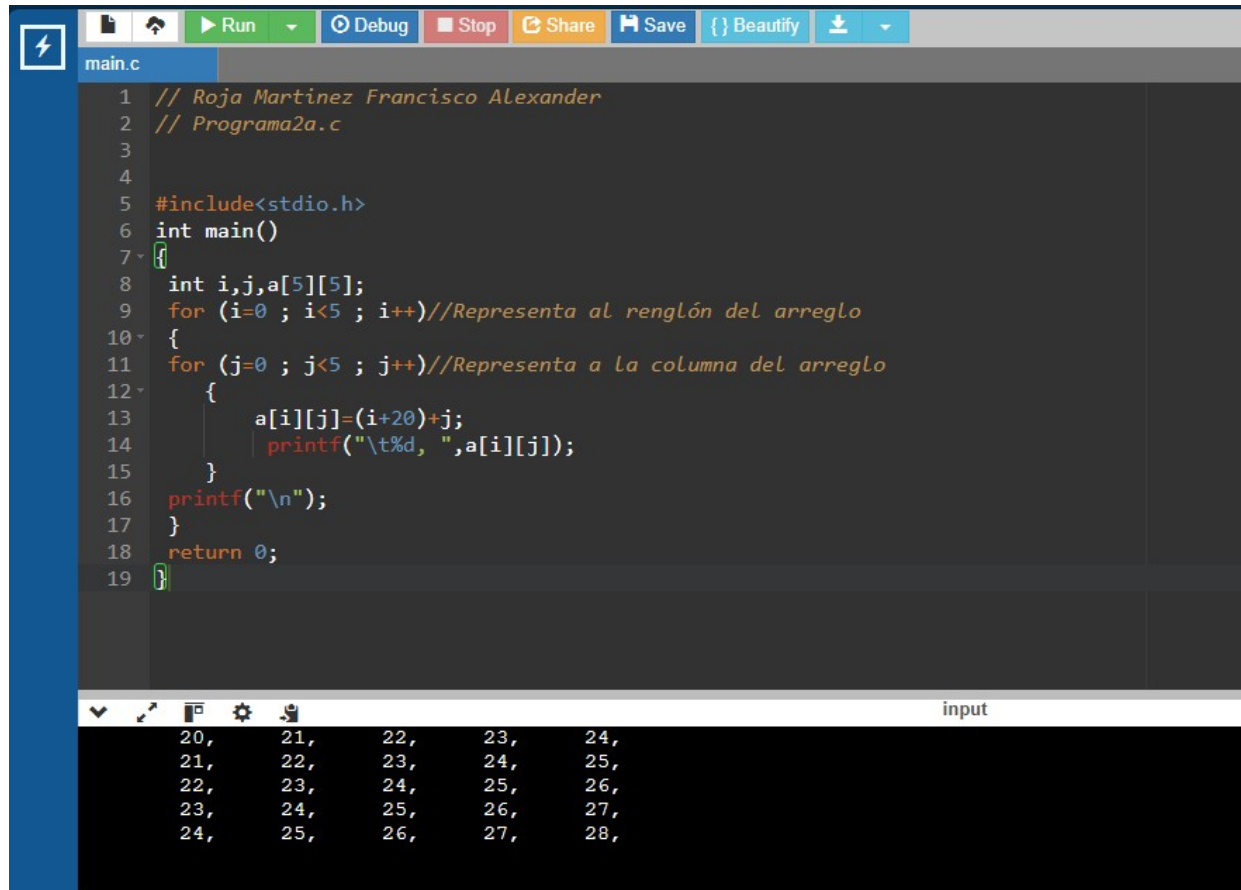
input

a posición [0][2]: Introduce el valor para la posición [1][0]: Introduce el valor para la posición [1][1]: Introduce el valor para la posición [1][2]: Introduce el valor para la posición [2][0]: Introduce el valor para la posición [2][1]: Introduce el valor para la posición [2][2]:

Imprimir Matriz

```
1 2 3
4 5 6
7 8 9
```

2.- Del programa 2a.c, modificar para que tome los valores a partir del numero 20, pueden utilizar diferentes formas



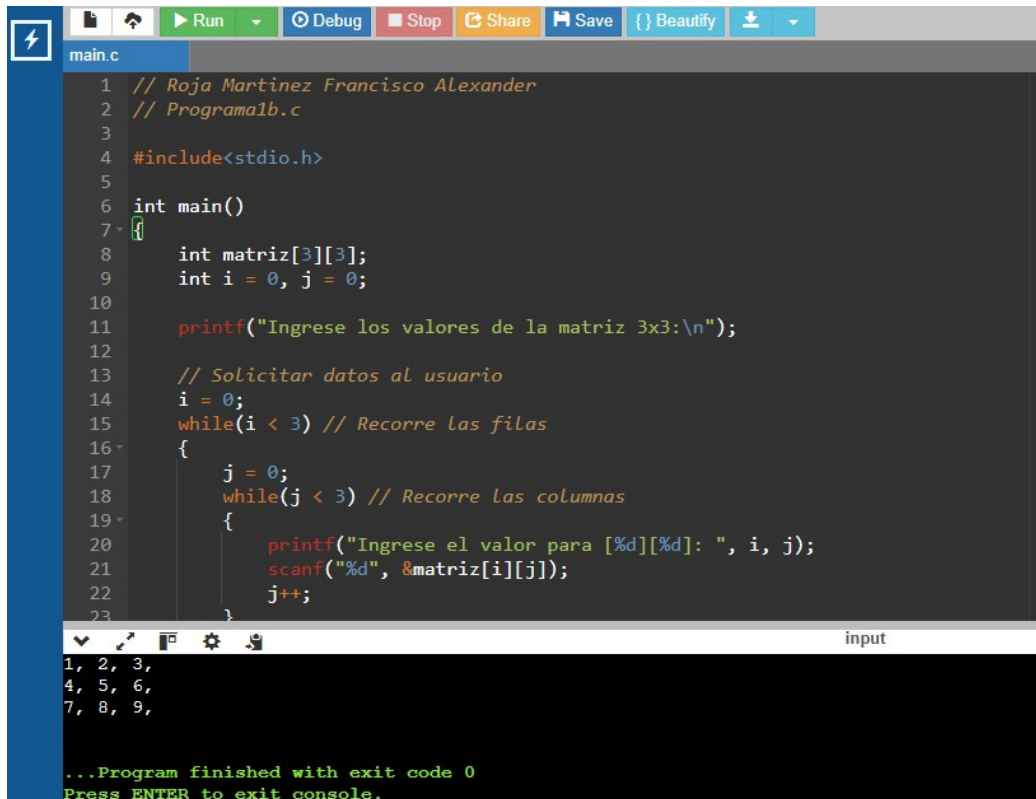
The screenshot shows a C program named `main.c` being executed. The code defines a 5x5 array `a` and fills it with values starting from 20. The output is a 5x5 grid of numbers, each followed by a comma, arranged in rows and columns.

```
1 // Roja Martinez Francisco Alexander
2 // Programa2a.c
3
4
5 #include<stdio.h>
6 int main()
7 {
8     int i,j,a[5][5];
9     for (i=0 ; i<5 ; i++)//Representa al renglón del arreglo
10    {
11        for (j=0 ; j<5 ; j++)//Representa a la columna del arreglo
12        {
13            a[i][j]=(i+20)+j;
14            printf("\t%d, ",a[i][j]);
15        }
16        printf("\n");
17    }
18    return 0;
19 }
```

input

20,	21,	22,	23,	24,
21,	22,	23,	24,	25,
22,	23,	24,	25,	26,
23,	24,	25,	26,	27,
24,	25,	26,	27,	28,

3.- Del programa 1b.c modificar para que solicite los datos al usuario con estructuras while



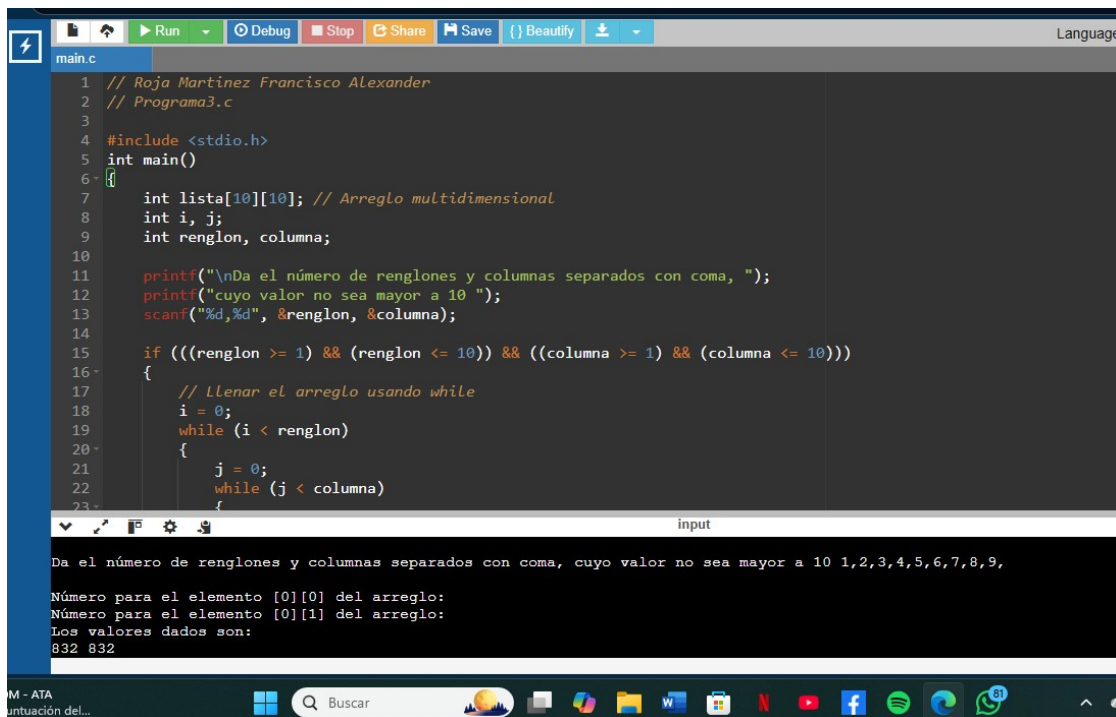
```
1 // Roja Martinez Francisco Alexander
2 // Programa1b.c
3
4 #include<stdio.h>
5
6 int main()
7 {
8     int matriz[3][3];
9     int i = 0, j = 0;
10
11     printf("Ingrese los valores de la matriz 3x3:\n");
12
13     // Solicitar datos al usuario
14     i = 0;
15     while(i < 3) // Recorre las filas
16     {
17         j = 0;
18         while(j < 3) // Recorre las columnas
19         {
20             printf("Ingrese el valor para [%d][%d]: ", i, j);
21             scanf("%d", &matriz[i][j]);
22             j++;
23         }
24         i++;
25     }
26 }
```

input

1, 2, 3,
4, 5, 6,
7, 8, 9,

...Program finished with exit code 0
Press ENTER to exit console.

4.- Del programa 3.c van a indicarme que realiza el programa y van a modificar todas las estructuras for sustituyendo por while o do-while



```
1 // Roja Martinez Francisco Alexander
2 // Programa3.c
3
4 #include <stdio.h>
5 int main()
6 {
7     int lista[10][10]; // Arreglo multidimensional
8     int i, j;
9     int renglon, columna;
10
11     printf("\nDa el número de renglones y columnas separados con coma, ");
12     printf("cuyo valor no sea mayor a 10 ");
13     scanf("%d,%d", &renglon, &columna);
14
15     if (((renglon >= 1) && (renglon <= 10)) && ((columna >= 1) && (columna <= 10)))
16     {
17         // Llenar el arreglo usando while
18         i = 0;
19         while (i < renglon)
20         {
21             j = 0;
22             while (j < columna)
23             {
24                 // ... (code for filling the array) ...
25             }
26             i++;
27         }
28     }
29 }
```

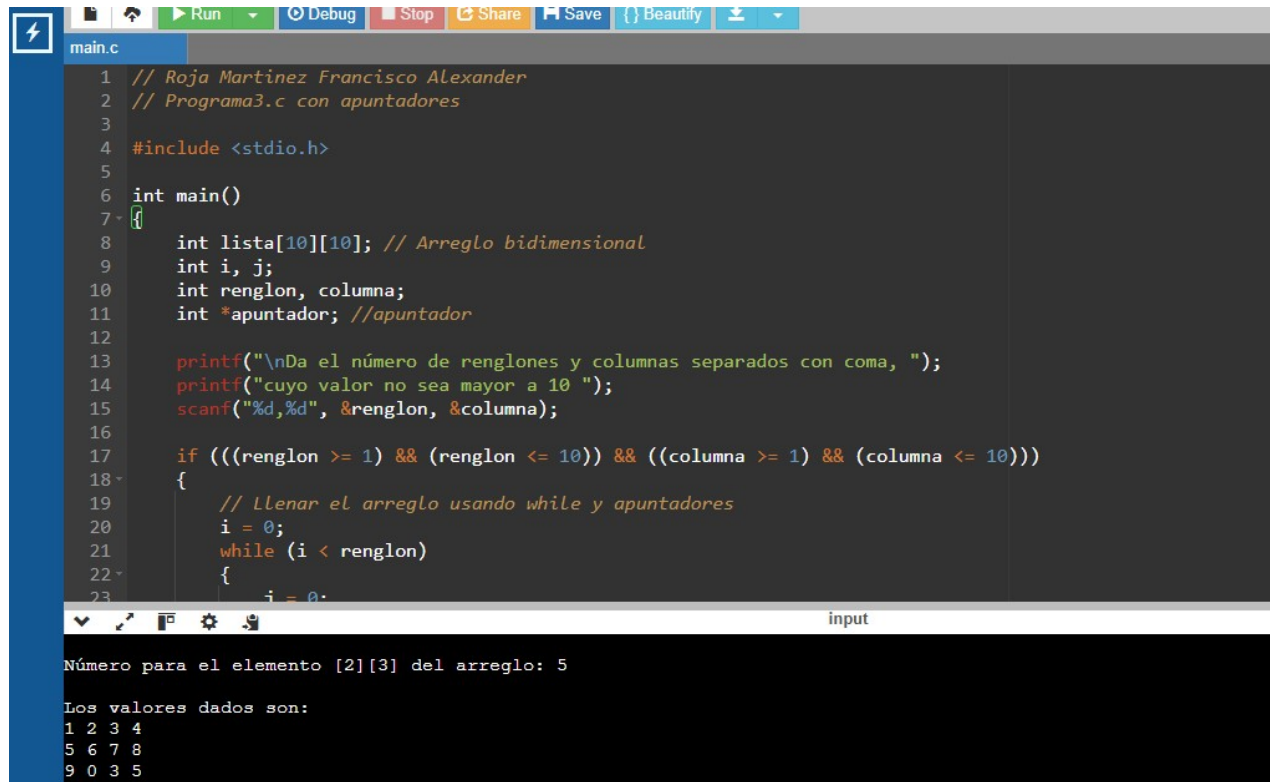
input

Da el número de renglones y columnas separados con coma, cuyo valor no sea mayor a 10 1,2,3,4,5,6,7,8,9,

Número para el elemento [0][0] del arreglo:
Número para el elemento [0][1] del arreglo:
Los valores dados son:
832 832

Este programa solicita al usuario los valores para crear la matriz y dependiendo de la multiplicación de rdos fod dependerá de los valores que solicitaran

5.- Del programa 3.c modificar aplicando apuntadores



```
1 // Roja Martinez Francisco Alexander
2 // Programa3.c con apuntadores
3
4 #include <stdio.h>
5
6 int main()
7 {
8     int lista[10][10]; // Arreglo bidimensional
9     int i, j;
10    int renglon, columna;
11    int *apuntador; //apuntador
12
13    printf("\nDa el número de renglones y columnas separados con coma, ");
14    printf("cuyo valor no sea mayor a 10 ");
15    scanf("%d,%d", &renglon, &columna);
16
17    if (((renglon >= 1) && (renglon <= 10)) && ((columna >= 1) && (columna <= 10)))
18    {
19        // Llenar el arreglo usando while y apuntadores
20        i = 0;
21        while (i < renglon)
22        {
23            j = 0;
24            while (j < columna)
25            {
26                *(lista[i] + j) = i * j;
27                j++;
28            }
29            i++;
30        }
31    }
32
33    printf("Número para el elemento [2][3] del arreglo: ");
34    scanf("%d", &apuntador);
35
36    printf("Los valores dados son:\n");
37    for (i = 0; i < renglon; i++)
38    {
39        for (j = 0; j < columna; j++)
40        {
41            printf("%d ", *(lista[i] + j));
42        }
43        printf("\n");
44    }
45}
```

input

Número para el elemento [2][3] del arreglo: 5

Los valores dados son:

```
1 2 3 4
5 6 7 8
9 0 3 5
```

Tarea

1.-Realiza un programa que muestre tu nombre y número de cuenta haciendo uso de 2 arreglos, emplear while y do-while

#include <stdio.h>

#include <string.h> // Necesario para la función strlen

int main() {

// 1. Declaración e inicialización de los arreglos de caracteres

```
// Arreglo para el Nombre CORREGIDO: Francisco Alexander Roja  
Martínez
```

```
char nombre[] = "Francisco Alexander Roja Martinez";
```

```
// Arreglo para el Número de Cuenta: 323191041
```

```
char num_cuenta[] = "323191041";
```

```
int i = 0; // Índice para el bucle 'while'
```

```
int j = 0; // Índice para el bucle 'do-while'
```

```
// -----
```

```
// PARTE 1: Mostrar el Nombre usando el bucle WHILE
```

```
// -----
```

```
printf("--- Mostrando Nombre con Bucle WHILE ---\n");
```

```
printf("Mi Nombre es: ");
```

```
// El bucle 'while' se ejecuta mientras el caracter actual NO sea el nulo ('\0').
```

```
while (nombre[i] != '\0') {

    printf("%c", nombre[i]);

    i++; // Avanzamos al siguiente carácter

}

printf("\n");

// -----

// PARTE 2: Mostrar el Número de Cuenta usando el bucle DO-WHILE

// -----

printf("\n--- Mostrando No. de Cuenta con Bucle DO-WHILE ---\n");

printf("Mi No. de Cuenta es: ");

int longitud_cuenta = strlen(num_cuenta);

if (longitud_cuenta > 0) {

    do {

        printf("%c", num_cuenta[j]);

        j++; // Avanzamos el índice
```

```

    } while (j < longitud_cuenta); // Condición de permanencia

} else {

    printf("[El arreglo de cuenta esta vacio]");

}

printf("\n");

return 0;

}

```

The screenshot shows a web-based GDB online debugger interface. The top bar indicates 'GDB online Debugger | Compiler - Code, Compile, Run, Debug online C, C++'. Below the bar is a toolbar with buttons for Run, Debug, Stop, Share, Save, Beautify, and a download icon. The language is set to 'C'. The code editor displays a file named 'main.c' with the following content:

```

1 //Roja Martinez Francisco Alexander
2 //Tarea 1
3
4 #include <stdio.h>
5 #include <string.h> // Necesario para la función strlen
6
7 int main() {
8     // 1. Declaración e inicialización de los arreglos de caracteres
9
10    // Arreglo para el Nombre CORREGIDO: Francisco Alexander Roja Martínez
11    char nombre[] = "Francisco Alexander Roja Martinez";
12
13    // Arreglo para el Número de Cuenta: 323191041
14    char num_cuenta[] = "323191041";
15
16    int i = 0; // Índice para el bucle 'while'
17    int j = 0; // Índice para el bucle 'do-while'
18
19    // -----
20    // PARTE 1: Mostrar el Nombre usando el bucle WHILE
21    // -----
22
23    printf("--- Mostrando Nombre con Bucle WHILE ---\n");

```

The output window at the bottom shows the execution results:

```

input
--- Mostrando Nombre con Bucle WHILE ---
Mi Nombre es: Francisco Alexander Roja Martinez

--- Mostrando No. de Cuenta con Bucle DO-WHILE ---
Mi No. de Cuenta es: 323191041

```

2.- Corrige e indica que realiza el siguiente programa:(Encuentra los 5 errores)

```
#include<studio.h>
```

```
main()
```

```
{
```

```
int fila,columna, cont=0,contadorApuntador;
```

```
float arreglo[3][3], acumulador=0, *ap;
```

```
ap = &arreglo[0][0];
```

```
for(fila=0; fila<3; fila++)
```

```
{
```

```
for(columna=0; columna<1;columna--)
```

```
{
```

```
printf("\t\n Teclear el elemento %d %d \t",  
fila+1,columna+1);
```

```
scanf("%f",&arreglo[fila][columna]);
```

```
acumulador+=arreglo[fila][columna];
```

```
}
```

```
}
```

```
printf("\tLa Matriz es:\n");
```

```
for (contadorApuntador=0 ; contadorApuntador<9 ;  
contadorApuntador++)
```

```
{
```

```
if (cont == 3)
```

```
{
```

```
printf("\n");
```

```
cont = 1;
```

```
}
```

```
printf("%.2f\t\t",*(ap+contadorApuntador));
```

```
cont++;
```

```
}
```

```
printf("\t\n\n La suma de los elementos es:%0.2d", acumulador);
```

```
return 0;
```

```
}
```

No.	Ubicación del Error	Error Original	Corrección
1	Encabezado	#include<studio.h>	#include <stdio.h>
2	Función Principal	main()	int main()
3	Bucle interno	for(columna=0; columna<1; columna--)	for(columna=0; columna<3; columna++)
4	Impresión de la suma	printf("...:%0.2d", acumulador);	printf("...:%0.2f", acumulador);
5	Lógica de salto de línea	if (cont == 3) \$\rightarrow\$ cont = 1;	if (cont == 3) \$\rightarrow\$ cont = 0;

```
#include <stdio.h> // CORRECCIÓN 1: studio.h -> stdio.h
```

```
int main() // CORRECCIÓN 2: main() -> int main()

{

    int fila, columna, cont = 0, contadorApuntador;

    float arreglo[3][3], acumulador = 0, *ap;


    // Asigna el apuntador 'ap' a la dirección del primer elemento (arreglo[0][0])

    ap = &arreglo[0][0];


    // Bucle para LECTURA de datos y ACUMULACIÓN de la suma

    for(fila = 0; fila < 3; fila++)

    {

        for(columna = 0; columna < 3; columna++) // CORRECCIÓN 3: <1 y
columna-- -> <3 y columna++

        {

            printf("\t\n Teclear el elemento (Fila %d, Columna %d): ", fila + 1,
columna + 1);


            // scanf utiliza &arreglo[fila][columna] para almacenar el valor en la
posición

            scanf("%f", &arreglo[fila][columna]);
```

```
        // Acumula el valor leído en la variable 'acumulador'

        acumulador += arreglo[filas][columna];

    }

}
```

// Bucle para IMPRESIÓN de la matriz usando apuntadores

```
printf("\t\nLa Matriz es:\n");
```

// Itera 9 veces (el total de elementos: 3*3)

```
for (contadorApuntador = 0; contadorApuntador < 9;
contadorApuntador++)
```

```
{
```

// Lógica de formato: Si 'cont' llega a 3, significa que se ha impreso una fila

```
if (cont == 3)
```

```
{
```

```
    printf("\n");
```

```
    cont = 1; // CORRECCIÓN 5: Resetear el contador a 1 (ya que el
cont++ lo llevará a 2)
```

```
}
```

// Imprime el valor: *(ap + contadorApuntador) accede al elemento k-ésimo desde el inicio

```
printf("%.2f\t\t", *(ap + contadorApuntador));
```

```
cont++;
```

```
}
```

// Imprime el resultado total de la suma de todos los elementos

```
printf("\t\n\n La suma de los elementos es: %.2f", acumulador); //
```

CORRECCIÓN 4: %0.2d -> %.2f

```
return 0;
```

```
}
```

```
main.c
1 //Roja Martinez Francisco Alexander
2 //Tarea 2
3
4 #include <stdio.h> // Incluye la librería estándar de entrada/salida
5
6 // Función principal del programa
7 int main()
8 {
9     // Declaración de variables para control de bucles y contadores
10    int fila, columna, cont = 0, contadorApuntador;
11
12    // Declaración de la matriz (3x3 de floats), el acumulador y el apuntador
13    float arreglo[3][3], acumulador = 0, *ap;
14
15    // Asigna el apuntador 'ap' a la dirección del primer elemento (arreglo[0][0])
16    // Esto permite acceder a toda la matriz de forma secuencial
17    ap = &arreglo[0][0];
18
19    // Bucle para LECTURA de datos y ACUMULACIÓN de la suma
20    for(fila = 0; fila < 3; fila++) // Itera por las 3 filas
21    {
22        for(columna = 0; columna < 3; columna++) // Itera por las 3 columnas
23        {
24            // Lectura de datos
25            printf("Introduce el valor de la fila %d y columna %d: ", fila, columna);
26            scanf("%f", ap);
27            *ap++;
28
29            // Acumulación de la suma
30            acumulador += *ap;
31        }
32    }
33
34    // Salida de la suma
35    printf("La suma de los elementos es: %f\n", acumulador);
36}
```

input

```
La Matriz introducida es:
1.00      4.00      7.00
4.00      2.00      9.00
7.00      4.00      9.00

La suma de los elementos es: 47.00
```

3.- Programa que solicite al usuario los valores de dos matrices de 3 x 3 y haga su multiplicación haciendo uso de arreglos. (Investigar como se realiza una multiplicación de matrices)

#include <stdio.h>

#define FILAS 3

#define COLUMNAS 3

// Función principal

```
int main() {  
  
    // 1. Declaración de las matrices y variables de control  
  
    float matriz_A[FILAS][COLUMNAS];  
  
    float matriz_B[FILAS][COLUMNAS];  
  
    float matriz_C[FILAS][COLUMNAS]; // Matriz resultado (C = A * B)  
  
    int i, j, k;  
  
    // -----  
  
    // PARTE 1: SOLICITAR VALORES PARA LAS MATRICES A y B  
  
    // -----  
  
    printf("--- INGRESO DE VALORES PARA MATRIZ A (3x3) ---\n");  
  
    for (i = 0; i < FILAS; i++) {  
  
        for (j = 0; j < COLUMNAS; j++) {  
  
            printf("Ingrese el elemento A[%d][%d]: ", i + 1, j + 1);  
  
            if (scanf("%f", &matriz_A[i][j]) != 1) {  
  
                printf("Error de lectura. Finalizando.\n");  
  
                return 1;  
  
            }  
  
        }  
  
    }  
  
}
```

```
    }  
}  
}
```

```
printf("\n--- INGRESO DE VALORES PARA MATRIZ B (3x3) ---\n");
```

```
for (i = 0; i < FILAS; i++) {
```

```
    for (j = 0; j < COLUMNAS; j++) {
```

```
        printf("Ingrese el elemento B[%d][%d]: ", i + 1, j + 1);
```

```
        if (scanf("%f", &matriz_B[i][j]) != 1) {
```

```
            printf("Error de lectura. Finalizando.\n");
```

```
            return 1;
```

```
        }
```

```
    }
```

```
}
```

```
// -----
```

```
// PARTE 2: MULTIPLICACIÓN DE MATRICES (A * B)
```

```
// -----
```

```
// Inicializar la matriz C a cero (necesario para la acumulación)
```

```
for (i = 0; i < FILAS; i++) {
```

```
    for (j = 0; j < COLUMNAS; j++) {
```

```
        matriz_C[i][j] = 0.0;
```

```
    }
```

```
}
```

```
// Triple bucle para la multiplicación: C[i][j] = Suma(A[i][k] * B[k][j])
```

```
for (i = 0; i < FILAS; i++) {    // Recorre las FILAS de A (i)
```

```
    for (j = 0; j < COLUMNAS; j++) {    // Recorre las COLUMNAS de B (j)
```

```
        for (k = 0; k < COLUMNAS; k++) { // Realiza el producto punto (k)
```

```
            matriz_C[i][j] += matriz_A[i][k] * matriz_B[k][j];
```

```
        }
```

```
    }
```

```
}
```

```
// -----
```

```
// PARTE 3: IMPRIMIR LA MATRIZ RESULTANTE C
```

```
// -----
```

```
printf("\n--- MATRIZ RESULTANTE C (A x B) ---\n");
```

```
for (i = 0; i < FILAS; i++) {
```

```
    for (j = 0; j < COLUMNAS; j++) {
```

```
        // Imprime cada elemento con 2 decimales y un espacio
```

```
        printf("%.2ft", matriz_C[i][j]);
```

```
    }
```

```
    printf("\n"); // Salto de línea al terminar cada fila
```

```
}
```

```
return 0;
```

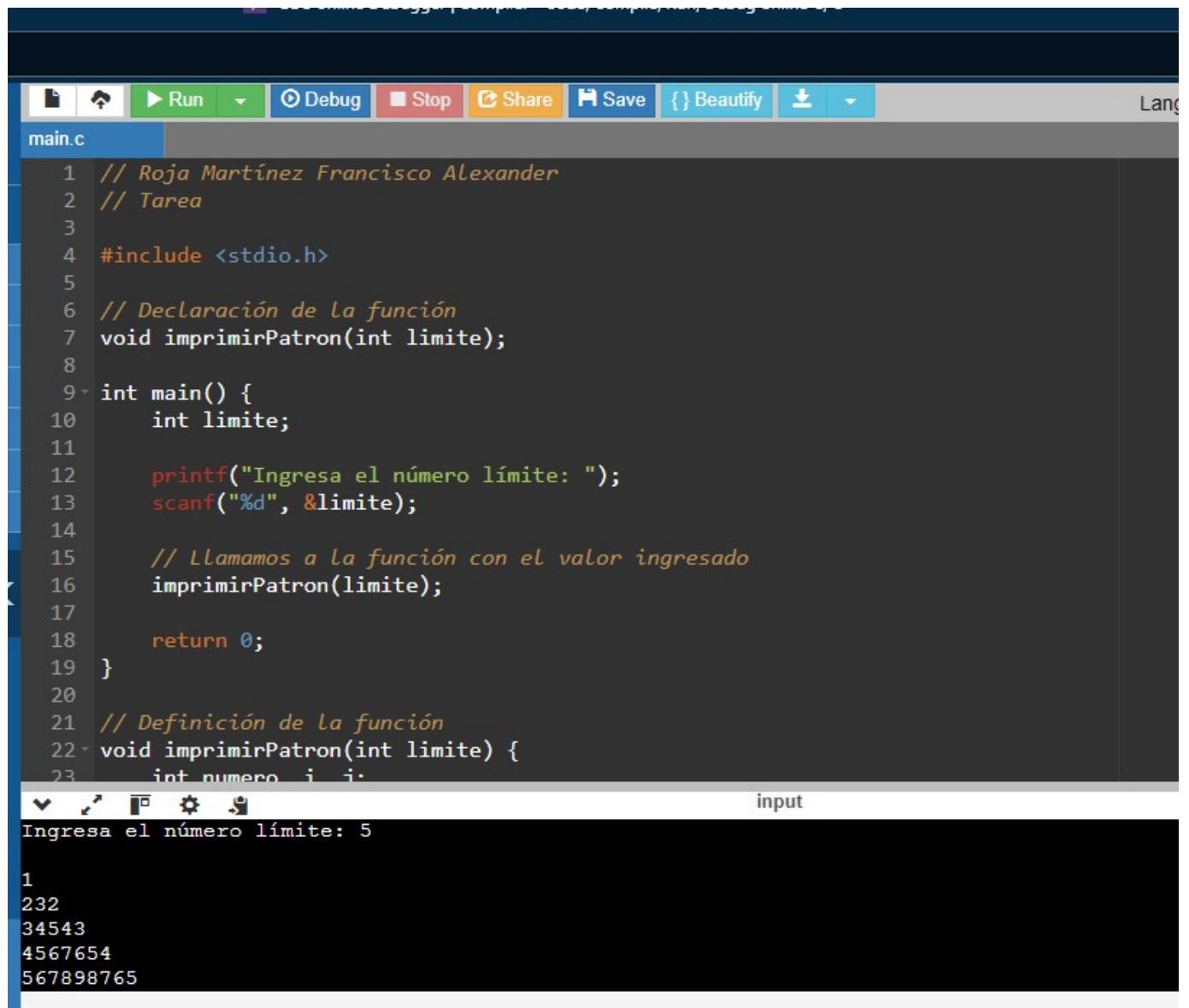
```
}
```

The screenshot shows a C program in a code editor with a terminal window. The program defines two 3x3 matrices, A and B, and calculates their product, matrix C. The terminal output shows the input for matrix A, matrix B, and the resulting matrix C. The program finishes with an exit code of 0.

```
main.c
1 //Roja Martinez Francisco Alexander
2 //Tarea 3
3
4 #include <stdio.h>
5
6 #define FILAS 3
7 #define COLUMNAS 3
8
9 // Función principal
10 int main() {
11     // 1. Declaración de las matrices y variables de control
12     float matriz_A[FILAS][COLUMNAS];
13     float matriz_B[FILAS][COLUMNAS];
14     float matriz_C[FILAS][COLUMNAS]; // Matriz resultado (C = A * B)
15
16     int i, j, k;
17
18     // -----
19     // PARTE 1: SOLICITAR VALORES PARA LAS MATRICES A y B
20     // -----
21
22     printf("--- INGRESO DE VALORES PARA MATRIZ A (3x3) ---\n");
23     for (i = 0; i < FILAS; i++) {
24         for (j = 0; j < COLUMNAS; j++) {
25             printf("Ingrese el elemento A[%d][%d]: ", i + 1, j + 1);
26             if (scanf("%f", &matriz_A[i][j]) != 1) {
27                 printf("Error de lectura. Finalizando.\n");
28                 return 1;
29             }
30         }
31     }
```

```
input
--- INGRESO DE VALORES PARA MATRIZ A (3x3) ---
Ingrese el elemento A[1][1]: 21
Ingrese el elemento A[1][2]: 11
Ingrese el elemento A[1][3]: 4
Ingrese el elemento A[2][1]: 1
Ingrese el elemento A[2][2]: 2
Ingrese el elemento A[2][3]: 5
Ingrese el elemento A[3][1]: 78
Ingrese el elemento A[3][2]: 5
Ingrese el elemento A[3][3]: 9
--- INGRESO DE VALORES PARA MATRIZ B (3x3) ---
Ingrese el elemento B[1][1]: 1
Ingrese el elemento B[1][2]: 4
Ingrese el elemento B[1][3]: 5
Ingrese el elemento B[2][1]: 8
Ingrese el elemento B[2][2]: 2
Ingrese el elemento B[2][3]: 3
Ingrese el elemento B[3][1]: 9
Ingrese el elemento B[3][2]: 1
Ingrese el elemento B[3][3]: 0
--- MATRIZ RESULTANTE C (A x B) ---
145.00 110.00 138.00
62.00 13.00 11.00
199.00 331.00 405.00
...Program finished with exit code 0
Press ENTER to exit console.
```

4.



```
main.c
1 // Roja Martínez Francisco Alexander
2 // Tarea
3
4 #include <stdio.h>
5
6 // Declaración de la función
7 void imprimirPatron(int limite);
8
9 int main() {
10     int limite;
11
12     printf("Ingresa el número límite: ");
13     scanf("%d", &limite);
14
15     // Llamamos a la función con el valor ingresado
16     imprimirPatron(limite);
17
18     return 0;
19 }
20
21 // Definición de la función
22 void imprimirPatron(int limite) {
23     int numero = 1;
24     while (numero <= limite) {
25         printf("%d", numero);
26         if (numero % 5 == 0) printf("\n");
27         numero++;
28     }
29 }
```

input

Ingresa el número límite: 5

1
232
34543
4567654
567898765

Conclusión

Esta serie de ejercicios y la discusión teórica han sido cruciales para consolidar y aplicar mis conocimientos fundamentales de estructuras de datos en C. La comprensión profunda de conceptos abordados en prácticas anteriores, como la manipulación básica de arreglos unidimensionales y el uso de los bucles (while, do-while), sin duda me facilitó significativamente la transición al manejo de estructuras más complejas, como los arreglos bidimensionales (matrices).

La actividad principal, centrada en la multiplicación de matrices 3×3 , fue particularmente enriquecedora, pues me obligó a integrar la lógica de los tres bucles anidados con la correcta gestión de índices para aplicar la fórmula del producto punto. Además, la exploración de la relación entre apuntadores y arreglos reafirmó mi entendimiento de cómo se organizan los datos de forma contigua en la memoria, permitiéndome el acceso eficiente mediante aritmética de punteros.

No obstante, reconozco que la sintaxis y las implicaciones de la aritmética de apuntadores en estructuras multidimensionales aún presentan cierta complejidad que requiere más práctica para dominar por completo.

Referencia

Facultad de Ingeniería. (2025). Manual de prácticas del laboratorio de Fundamentos de Programación. Laboratorio de computación. Salas A y B. Universidad Nacional Autónoma de México. pp.149-161 . Recuperado el 21 de octubre de 2025 de [Laboratorio Salas A y B](#)